An Empirical Analysis of Multi-Agent Systems for Football Data Analytics and Performance Prediction

Detailed 10-page study summary - simplified language, maximum useful detail

Source paper: LSE ST498 Capstone Project, Department of Statistics, 2025/26. This summary follows the report's own structure, experiments, terminology, and caveats.

1. What the project is trying to discover

The project asks a practical AI-systems question: when a user wants to analyse football data conversationally, is it worth using a Multi-Agent System (MAS), or can a single Large Language Model with the right tools do the same job more cheaply and quickly? Football is a useful test domain because the underlying evidence is rich and structured - match results, league tables, player statistics, events, xG, transfers, injuries and odds - while user questions can range from simple lookups to comparisons, aggregations and forecasts.

The starting intuition comes from multi-agent research such as AutoGen and MetaGPT. A MAS decomposes a task across specialised components instead of asking one model to plan, retrieve, calculate, validate and explain in one pass. In this project, a Manager interprets intent; Data Discovery selects relevant data; a Data Analyst writes SQL; a Prediction Agent calls specialised models; a Synthesizer writes the answer; a Visualizer can generate visual output; and an Eval Agent checks consistency. In theory, specialisation can narrow each agent's context, tool use can replace hallucinated computation, and errors can be isolated before they propagate.

The authors argue, however, that many existing MAS benchmarks make it difficult to know why a multi-agent system wins. Often the MAS is also given better tools, retrieval or database access. If it then scores higher, the gain may come from the tools rather than from orchestration itself. This motivates a controlled comparison in which tool access and schema context are held constant where possible.

Four research questions

1. Autonomy: How does the Data Analyst perform when it is controlled by templates, allowed to mix templates with free SQL, or fully autonomous?
2. Orchestration vs tools: Once both systems have the same database tools and schema context, does a MAS add accuracy beyond a single tool-using LLM?
3. Memory: Does persistent memory improve accuracy, latency or token cost, especially in multi-turn sessions?
4. Prediction: Can the system delegate a forecast to a calibrated match-outcome model and return reliable probability distributions rather than free-text guesses?

What counts as success

The paper deliberately does not look only at a headline accuracy score. It measures accuracy together with token usage and inference time, because a system that gains one percentage point while doubling cost may not be a good production choice. It also separates deterministic data-analysis questions from probabilistic forecasting questions because these stress different parts of the architecture.

Core framing: the paper is not trying to prove that MAS are always better. It is trying to identify when extra orchestration is actually justified.

2. Data sources and the integrated football knowledge base

A large part of the engineering effort is devoted to the data layer. The operational knowledge base spans twelve seasons, from 2014-15 to 2025-26, across ten European top-flight competitions: Premier League, La Liga, Bundesliga, Serie A, Ligue 1, Eredivisie, Primeira Liga, Belgian Pro League, Turkish Super Lig and Scottish Premiership. It covers roughly six thousand players. The database is intentionally broader than any one experiment because different analytical capabilities need different observational levels.

Provider roles

- WhoScored: event-level actions such as passes, shots, tackles, dribbles, fouls, duels, set pieces and goalkeeper actions, with time, coordinates, outcomes and provider-specific qualifiers.
- Understat: attacking and possession-chain metrics at shot, player-match and player-season level, including goals, assists, minutes, shots, xG, xA, non-penalty xG, xGChain and xGBuildup.
- FBref: match reports, line-ups, formations, substitutions, referee and venue metadata, outfield and goalkeeper tables, league standings, home/away splits and season-level team aggregates.
- Transfermarkt: player profiles, market-value history, transfers, transfer fees and injury spells, including time-varying observations such as days or games missed.
- Football-Data: match-level validation statistics and bookmaker odds. Decimal odds are converted into implied probabilities and normalised to remove bookmaker overround.
- World Cup supplement from Kaggle: a limited international-tournament extension processed through the same cleaning and standardisation framework.

The challenge is not simply data volume. The difficult part is that providers use incompatible identifiers, names, schemas, granularities and update schedules. The same club or player can appear under different names and IDs; date formats and column structures differ; some sources are nested or semi-structured. The project therefore resolves these problems upstream rather than asking an LLM to improvise reconciliation at query time.

Five-stage ETL pipeline

1. Raw extraction and staging: each provider is collected separately and stored unchanged. Original columns, provider IDs, nested structures, source files and provenance metadata are preserved so transformations can be reproduced without re-scraping.
2. Source-specific cleaning: columns are renamed consistently, temporal fields are parsed, competition and season labels are standardised, monetary/numeric fields are cast, duplicate records are removed, and provider-specific structures are flattened or reshaped.
3. Entity resolution: teams, players, matches, competitions and seasons are mapped to canonical internal entities. Team aliases use deterministic dictionaries; player matching combines cleaned names, transliteration, fuzzy similarity and contextual checks such as nationality, birth year, club and season.
4. Validation and database loading: every transformed frame is checked against a schema contract for required columns, types, key uniqueness, foreign-key coverage, valid categorical values and plausible ranges. Match-level consistency and child-parent links are also checked.
5. Serving tables and model-ready features: materialised views pre-compute frequent joins and aggregations. Time-varying features are aligned to the analytical date to prevent information leakage.

Ambiguous entity mappings are logged for manual review rather than silently accepted. Players appearing in events before a full profile exists can be inserted as stub entities, preserving referential integrity until later enrichment. The loader uses idempotent upserts and explicit conflict keys, so rerunning it should not create duplicates.

Data philosophy: messy provider-specific quirks should be absorbed during ingestion and standardisation, not exposed to the agents that answer user questions.

3. Three-layer database design and YAML schema documentation

Three relational layers

The knowledge base is implemented on managed PostgreSQL and divided conceptually into raw, canonical and serving layers. The raw layer preserves provider-native records for audit and reprocessing. The canonical layer stores normalised competitions, seasons, teams, players, matches and related fact tables under stable internal IDs. The serving layer exposes stable, semantically meaningful relations that are easier and safer for agents to query.

Principal serving views include match summaries, player-match summaries, player-season summaries, shooting profiles, availability histories and team-season summaries. The purpose is to avoid forcing an agent to rebuild complex provider-specific joins for every question. A user asks about football concepts; the serving layer should expose football concepts rather than source plumbing.

Observational grain is treated explicitly

- match table: one row per match;
- event table: one row per event;
- player-match table: one row per player and match;
- player-season table: one row per player, team and season;
- team-season tables: one row per team and season;
- market values: one row per dated observation;
- transfers: one row per transfer;
- injuries: one row per injury spell.

This matters because joins across different grains can silently duplicate observations. Joining one match directly to hundreds of events multiplies the match row; joining one player-season row to several transfers can create multiple copies of the same seasonal statistics. The database design and schema documentation therefore guide the agent toward aggregating or filtering each fact table before combining grains.

Temporal integrity and leakage prevention

For predictive features, all time-varying information must be known before the target match. Rolling variables are ordered chronologically and shifted so the current match cannot influence its own features. Market values, transfers, injuries, recent form, rest periods and availability use observations on or before the relevant date. This turns leakage prevention into an engineering rule rather than a modelling afterthought.

Machine-readable YAML schema

The SQL schema is accompanied by a YAML specification injected into the language-model context. It documents each relation's purpose, grain, primary and foreign keys, important columns, permitted joins, temporal meaning and caveats. This gives the model semantic knowledge of the database rather than only a raw list of tables and columns.

The YAML has two main functions. First, it improves SQL generation: the model is told what each relation means and how it may be joined, so it does not need to reverse-engineer the database. Second, it supports validation because rule-based checks can use the same expected grain and join paths. The YAML is maintained alongside the physical SQL schema, and structural changes should be reflected in both so the model is not given stale metadata.

Why this becomes central to the final conclusion

The experiments ultimately suggest that the quality of structured data access is more important for accuracy than sophisticated orchestration on the tested workload. This is why the report ends with an engineering recommendation: before adding more agents, invest in canonical entities, stable serving relations, explicit joins and well-maintained schema documentation.

A documented schema is treated as authoritative database knowledge; conversation history, SQL cache and corrections are separate memory mechanisms built from prior interactions.

4. Multi-agent architecture, control flow and SQL self-debate

LangGraph represents the MAS as a directed graph: nodes are specialised agents or functions, edges define control flow, and a shared state object carries structured information through the turn. The graph makes routing explicit and inspectable, which is one of the engineering benefits the authors still value even when accuracy does not improve.

Main query path

1. Memory Retrieval runs first and retrieves relevant conversation, cached SQL, past corrections and static documents.
2. Manager classifies the request and extracts structured parameters such as intent, teams, seasons, metrics and match date.
3. Deterministic data questions go to Data Discovery and Data Analyst; forecast questions go to the Prediction Agent; conversational or error cases can route directly toward synthesis.
4. Data Discovery identifies relevant tables and provides join/grain guidance.
5. Data Analyst generates SQL under a read-only guardrail.
6. Visualizer may be called for a full analytical pipeline.
7. Synthesizer writes the user-facing answer using the retrieved result.
8. Eval Agent performs rule-based consistency checks without another LLM call.
9. Memory Store writes successful turns, reusable SQL or corrections before the response returns.

Manager output

The Manager does not solve the question. It converts the natural-language request into structured routing information. The prompt asks for an intent such as data_request, prediction_match or conversational, together with seasons, teams, metrics and, for a forecast, home team, away team and match date. This makes downstream behaviour easier to trace than if one model simply improvised a complete plan.

Autonomous SQL self-debate

The fully autonomous Data Analyst uses a concrete self-debate mechanism rather than a vague self-critique instruction. Two independent candidate SQL statements are generated for the same question. They are executed, and their result sets are compared. If the results disagree, the model is shown both attempts and explicitly told that they conflict. It then reconciles them and produces a revised query. There are up to two reconciliation rounds; if agreement is not reached, the answer can be flagged low confidence.

The two candidates are prompted to be independent, including different phrasing or joins where appropriate. This is intended to catch semantic SQL mistakes that a syntax checker would miss. PostgreSQL can determine whether a query runs; the debate is supposed to help determine whether the query actually answers the user's question.

Bounded retry and safety

The retry process is bounded because autonomy has a cost. An unconstrained loop could repeatedly call the LLM, consume tokens and add latency without converging. The system therefore grants limited reconciliation effort rather than allowing indefinite retries. Database execution is also protected by a read-only validation layer: free SQL is permitted only for analytical SELECT statements, while modification statements are rejected.

Synthesis and evaluation

The Synthesizer receives the user question plus the retrieved result and is instructed to answer directly using only that evidence. The rule-based Eval Agent then compares the natural-language response against the retrieved figures. If the answer says 20 goals while the data says 15, the inconsistency can be flagged and written to correction memory. This final checker adds no model call, which keeps its cost small.

Engineering value of the graph: each failure can be localised to routing, discovery, SQL, prediction, synthesis or evaluation instead of being hidden inside one long monolithic prompt.

5. Memory architecture: conversation, SQL cache, corrections and RAG

The system is stateful across turns using PostgreSQL with pgvector. Text is embedded locally with all-MiniLM-L6-v2 into 384-dimensional vectors, stored with an HNSW index, and retrieved using cosine similarity. The report distinguishes agent memory from static RAG knowledge and from temporary working memory.

Four pgvector stores and thresholds

- conversation_memory - keyed by user question, written when a turn succeeds, threshold 0.70;
- sql_cache - keyed by user question, written when SQL returns rows, threshold 0.88;
- correction_memory - keyed by user question, written when a turn errors or is flagged, threshold 0.75;
- documents - keyed by document text, populated offline for RAG, threshold 0.30.

The same embedding model is used across all four stores so their vector space is consistent. Question-keyed stores compare a new question against previous questions. The documents store is different: the query is compared directly against static text such as player profiles or match reports. The agents do not write to the documents store during a turn.

Retrieval happens before intent classification

A dedicated retrieval node runs before the Manager. Relevant prior turns can therefore influence routing and synthesis. A past question about Arsenal's 2023-24 points may be resurfaced when a later user asks how that season compares with the previous one. A correction hit can warn the Data Analyst about a previously observed failure. A document hit can add qualitative context.

SQL cache as an efficiency mechanism

A cache hit can bypass SQL generation altogether. If a new question closely matches a previously successful question, the system may reuse the stored SQL. This can save an expensive generation/self-debate step. However, false positives are dangerous: two questions may be almost identical in embedding space but differ in season, team or metric, which changes the correct answer completely.

The deployed cache therefore uses a two-stage gate. First, cosine similarity must exceed 0.88. Second, a lightweight consistency guard compares discriminating tokens such as season, metric family and named entities between the incoming query, cached question and stored SQL. Any contradiction rejects the cache reuse. The guard is textual rather than another LLM call, so it adds negligible cost.

Correction memory

Correction memory records failed patterns and their fixes. The Eval Agent can write a correction when a response contradicts retrieved data. A prediction-outcome hook can also record a realised mismatch when a later outcome becomes available. On future similar queries, the correction is surfaced as cautionary context so the system is less likely to repeat the same mistake.

Working memory and session history

LangGraph's per-turn state acts as working memory: incoming question, retrieved memories, parsed entities, intermediate outputs and SQL results are carried through the graph and discarded after the turn. The report also notes a session-level transcript variable that accumulates messages but is write-only from the agents' perspective in the deployed design; it is not read into graph decisions and resets with a new chat.

Auxiliary logging

Two additional tables, feedback and trajectory_log, collect signals for offline analysis but are not read by agents during a turn. This separates online decision context from observability and later evaluation.

Memory is not assumed to improve every answer. The experiments explicitly test whether each store contributes accuracy, cost reduction, or session continuity.

6. Prediction node and the calibrated match-outcome model

Prediction is treated as a specialised capability rather than as free-text LLM reasoning. When the Manager recognises a fixture forecast, the query is routed to a Prediction Agent that resolves the teams and date, selects a capability and calls a pre-trained model artifact. The prediction package is deliberately decoupled from LangGraph so it can be trained, evaluated or replaced independently of the agent layer.

Capability routing

Thirteen prediction capabilities exist at inference, grouped into six families: match outcome; team-season parametric models; player parametric models; deterministic composite indices; counterfactual/similarity reuse; and empirical-frequency estimates. Selection uses a cascade: first the Manager's task identifier is checked, then a keyword-based fallback is tried, and finally a fixed default is used if neither resolves cleanly. The purpose is interpretability and graceful degradation when a free-text LLM label is malformed.

Only the match-outcome capability is fully detailed and externally benchmarked in the report. Other trained families are illustrative on the current sample, while deterministic or similarity-based families are assessed mainly for routing correctness and plausibility rather than competitive predictive accuracy.

Three-class outcome model

Match outcome is framed as Home win, Draw or Away win. Three base learners are trained: multinomial logistic regression, XGBoost and LightGBM. Each produces a three-probability vector, so together they generate nine values. These nine probabilities are passed to a logistic-regression meta-learner. This is stacking: the meta-model learns how to combine base models that make different errors rather than simply averaging them.

Out-of-fold stacking

To stop the meta-learner from training on overly optimistic in-sample predictions, base-model probabilities are generated through 5-fold cross-validation. Each base learner trains on four folds and predicts the held-out fifth. Every meta-feature vector therefore comes from a model that did not train on that match.

Isotonic calibration

The stacked probabilities are recalibrated per class using isotonic regression and renormalised to sum to one. Calibration matters because a model may rank outcomes correctly but still be overconfident. If a model repeatedly says 70%, roughly 70% of such events should occur. This is essential when users compare system probabilities with bookmaker odds: the magnitude of the probability must be trustworthy, not only the ordering of outcomes.

Features and leakage controls

- team-strength Elo;
- rolling five-match points and form;
- goals for and against;
- shots and xG for/against;
- rest days;
- team-level xG aggregated from player data.

All features are strictly pre-kickoff. The paper trains a no-odds model for live use and a with-odds model for offline comparison. At inference, a synthetic future fixture is appended to the historical match record and passed through the same feature-engineering pipeline used during training. If the resulting feature vector is incomplete, the runtime refuses to predict rather than silently imputing missing values.

Train/test design

The match-outcome model is trained on Premier League matches from 2020-2024, totalling 1,502 fixtures, and evaluated chronologically on the held-out 2024-2025 season, 379 fixtures. Baselines are training-set class frequencies and bookmaker-implied probabilities normalised to remove overround.

Prediction contribution: the agent system delegates forecasting to a causal, calibrated model instead of inventing probabilities in natural language.

7. Experimental design and benchmark construction

Two single-LLM baselines

Experiment 2 compares the autonomous MAS against two single-model conditions stripped of multi-agent orchestration. Both use the same short role description and schema context. The single-LLM-with-tools condition has native function-calling access to run_sql and predict_match_outcome, matching the MAS tool set. The raw-data condition has no tools or database connection; instead, a fixed dump of core tables is inlined into its prompt.

The raw-data tables include standings, home/away splits, team discipline and player-season statistics. Very large or specialised sources - events, player-match performance, transfers, market values and injuries - are excluded because of context-window size. Therefore only 20 of the 30 main data-analysis questions are answerable in this condition. This design choice later becomes an explicit limitation because data coverage and tool access are confounded.

Ground truth and freezing

The main head-to-head benchmark contains 60 Premier League questions: 30 deterministic data-analysis questions and 30 forecast questions. Each analysis question is paired with a hand-verified canonical SQL query whose output is treated as the reference answer. Questions, SQL and results are content-hashed and frozen together with the originating Git commit. This prevents benchmark drift: if a score changes, the authors can be confident that the benchmark itself did not silently change between runs.

Data-analysis scoring is programmatic. Scalars use exact comparison with a small numeric tolerance; tabular and ranked outputs use row-set comparison. An LLM-as-judge exists for genuinely free-text cases but is not needed for this golden set because every item reduces to deterministic SQL or a numeric forecast score.

Question coverage

The 30 analytical questions span nine categories and four difficulty levels. Examples include league winners and points, full tables, goal difference, point deductions, top scorers and assisters, home/away splits, yellow and red cards, referee queries, xG over/underperformance, transfers, injuries, single-match player performance and edge cases such as asking for Norwich City points in a season where the correct behaviour is denial.

Prediction set

The 30 forecast fixtures span August 2024 to May 2025 across all 20 Premier League clubs, with 10 home wins, 9 draws and 11 away wins. They are scored both on Brier score and on fidelity: whether the system correctly delegates to the prediction model and reproduces the returned probability vector.

Experiment 1 - autonomy

The earlier autonomy experiment uses the original 46-question single-league set. It compares controlled, semi-controlled and autonomous Data Analyst variants while holding Manager, Data Discovery, Synthesizer and schema context fixed. The set is divided into 28 in-coverage questions answerable by existing templates and 18 out-of-coverage questions requiring different query or join logic.

Experiment 3 - memory

Memory is evaluated on 55 short ordered sessions because independent questions give memory nothing useful to retrieve. There are 20 correction-memory sessions, 18 SQL-cache sessions and 17 conversation-memory sessions. Every session is run with memory on and off under otherwise identical architecture, creating a matched ablation. A separate cache stress test uses exact repeats, light rewords and near misses.

Controlled comparison logic: single raw -> single tools estimates the effect of scalable data/tool access; single tools -> MAS estimates the additional effect of orchestration.

8. Results: autonomy and MAS versus single LLM

Autonomy experiment

Templates are highly reliable when the question lies inside their catalogue. The controlled variant scores 26/28, or 93%, on in-coverage questions for all three frontier models tested: Claude Opus 4.8, GPT-5.5 and Gemini 3.1 Pro Preview. But the same controlled design collapses out of coverage because templates cannot express many computed columns, HAVING filters, cross-season aggregations or specialised event/transfer/injury logic. Out-of-coverage accuracy is only 6% for Opus, 17% for GPT and 11% for Gemini.

Free SQL recovers this flexibility. The autonomous Opus configuration reaches 17/18, or 94%, on out-of-coverage questions - the strongest such result. But it is less reliable on easy in-coverage questions: Opus autonomous falls to 85% there and GPT to 82%. The reason is that free SQL has more ways to be subtly wrong in formatting, ordering, aggregation or joins than a known-good template.

Model sensitivity is especially visible for Gemini. Its autonomous variant reaches only 70% overall, whereas its semi-controlled version reaches 91%. The report notes frequent failure of Gemini's autonomous self-debate loop to converge. This means the same orchestration code can behave very differently after a foundation-model swap, an important production concern for teams that may change providers for cost or availability.

Although semi-controlled achieves the highest overall score in some cases, the subsequent experiment uses fully autonomous Opus because the project prioritises robustness to unseen and ambiguous questions rather than performance only on known template types.

Main head-to-head: data-analysis queries

On the re-scoped 30-question multi-league benchmark, the autonomous MAS and the single LLM with the same tools tie exactly at 26/30, or 87%. The cost difference is large. The MAS averages 32,324 tokens and 16.7 seconds; the single tool-using LLM averages 22,150 tokens and 8.1 seconds. The MAS therefore uses roughly 1.5 times the tokens and about twice the latency without measured accuracy gain.

The paired McNemar exact test finds no significant difference, $p = 1.0$. The systems disagree on only four questions: two MAS-only correct and two single-only correct under the automatic scorer. Manual inspection shows that two disagreements are scorer artefacts, including a transfer fee formatted as "116.6 million" instead of 116600000 and a correct denial affected by markdown emphasis. The remaining genuine disagreements are both MAS errors. So even the programmatic 87%-87% tie slightly flatters the MAS.

On the 12 out-of-coverage questions in this re-scoped set, the single LLM with tools actually scores 12/12 while the MAS scores 10/12. The sample is too small to prove a modest true difference, but it provides no evidence for a material orchestration advantage on this workload.

Raw-data baseline and the coverage issue

The raw-data LLM scores 21/30, or 70%, and uses 121,711 tokens on average because the inlined tables must be re-read on every call. Crucially, on the 20 questions whose required tables are present, it scores 20/20. Its nine-point shortfall is driven by questions whose data are absent by construction. Therefore the 70%-87% gap should not be interpreted simply as "tools make the model reason better"; it mostly demonstrates that raw in-context data does not scale to full database coverage.

Headline result: on mostly single-step analytical questions, orchestration adds no measurable accuracy once the single LLM receives equivalent tools and schema context.

9. Results: memory and prediction

SQL-cache stress test

The cache experiment first runs the 46-question set cold, then as exact repeats, light rewords and near misses. Cold execution averages 20,656 tokens and 14.44 seconds. Exact repeats hit the cache 45/46 times, reduce average tokens to 11,575 and latency to 11.97 seconds, while preserving the same 72% accuracy as the cold baseline. That is roughly 44% fewer tokens and 17% lower latency because SQL generation is skipped entirely.

Light rewords hit the cache 44/46 times, but three answers drift and accuracy falls to 65%. More seriously, near misses - questions with almost the same surface wording but a swapped player or season - hit 43/46 times and accuracy collapses to about 7%. Average similarity remains around 0.978, far above the 0.88 threshold. The experiment demonstrates that embeddings are useful for finding similar questions but unsafe as the sole decision rule for reusing SQL in temporal or entity-sensitive analytics.

A later deployed guard checks season, metric family and named entities before reuse and rejects any contradiction. The report is careful to note that the original near-miss sweep predates this guard, so its end-to-end effectiveness on the same stress test has not yet been demonstrated. It also does not automatically catch polarity flips such as "highest" versus "lowest" when entities and metric remain the same.

Session-based memory ablation

Correction memory shows a +0.200 paired accuracy change on beneficiary turns, with bootstrap 95% interval [0.05, 0.40], but Wilcoxon p = 0.125 because only a few sessions have non-zero differences. In practice, the main recurring failure involves referee names containing hidden formatting characters. In four of five such recurring failures, the memory-enabled arm stores and reapplies a normalising pattern, while one case misses retrieval.

Conversation memory has exactly zero net accuracy change against the matched MAS without memory: five sessions improve, five degrade and seven remain unchanged. Against a completely stateless single LLM with no history it performs much better (+0.441), but it is beaten by a single LLM given the full conversation history (-0.559 relative difference). The full-history solution is more accurate for reference resolution but much more expensive in tokens, so retrieval-based memory is justified primarily by cost and scalability.

SQL cache shows no accuracy change in the 18 session tests. This reinforces the paper's revised memory conclusion: caching earns its place by reducing repeated computation, not by making answers intrinsically more correct.

Prediction delegation

The MAS and the single LLM with tools both delegate correctly on all 30 forecast questions and reproduce exactly the same model probabilities. Their Brier score is therefore identical at 0.197 by construction. The raw-data LLM cannot call the model and instead reasons from recent results, obtaining Brier 0.196 on this small set. The report does not treat this as evidence against the model because 30 fixtures are few and public Premier League results may be partly memorised by the LLM.

Full 379-match backtest

The calibrated no-odds model achieves 53.3% accuracy and Brier 0.199, compared with 40.9% and 0.218 for the base-rate baseline. The with-odds model reaches 53.6% and 0.194, while market-implied probabilities reach 54.1% and 0.193. Thus the no-odds model improves accuracy by 12.4 points over base rate and lands close to the market despite using no odds. Permutation analysis identifies Elo as the dominant feature group: shuffling Elo costs about 9.4 accuracy points, compared with 2.5 for defensive form and 1.2 for attacking form.

A major weakness remains draws. Under argmax, the model never makes draw the top class on the held-out season, even though draws are roughly a quarter of fixtures. This is one reason the authors recommend communicating the full calibrated probability distribution rather than only the single most likely label.

Memory finding: useful chiefly for reuse, continuity and avoiding repeated mistakes. Prediction finding: specialised models can be cleanly delegated to, but the agent architecture itself does not improve the forecast once both systems call the same model.

10. Limitations, interpretation and practical recommendations

Most important limitation: the task does not strongly exercise orchestration

The benchmark is dominated by single-step analytical queries in which one retrieval leads to one answer. Multi-agent orchestration is theorised to help most on genuinely multi-step tasks: decomposition into dependent sub-queries, intermediate results feeding later reasoning, planning and revision across stages. The null result therefore applies to this workload, not to all possible MAS. The paper identifies a multi-step compositional benchmark as the most important next experiment.

Tool access and data coverage are confounded

The raw-data condition cannot contain the full database because of context-window limits. Since it scores 20/20 on questions whose data are present, the 70%-87% overall difference mostly reflects missing coverage rather than clearly better reasoning from tool use. A cleaner future baseline would give a non-agent retrieval or text-to-SQL system equivalent data coverage without multi-agent orchestration.

The architecture is a static pipeline

The implemented MAS is a fixed LangGraph pipeline with specialised stages. It does not dynamically spawn many agents, negotiate, redesign the control flow or revisit arbitrary nodes based on intermediate state beyond the bounded retry. Therefore the results concern pipeline-style orchestration, not all dynamic or decentralised multi-agent architectures.

Small benchmark and single-run uncertainty

The main data-analysis benchmark has only 30 questions and each condition is effectively a single run. The autonomous analyst is stochastic, so one-to-three-question variation is plausible. McNemar's test has only a few discordant pairs and is underpowered: it rules out evidence for a large advantage but cannot exclude a small true effect. Repeated runs and a benchmark roughly an order of magnitude larger would be needed for precise differences.

Prediction and database limitations

The prediction package is separate from the orchestration question: tool-using MAS and single LLM produce identical forecasts once they call the same function. The 30-fixture comparison against raw LLM reasoning is too small and memorisable to establish model value. The main backtest is also confined to one Premier League season, and argmax never predicts draws. Separately, the database substrate changed during the project: the autonomy experiment uses the original single-league setup, while the later comparison is re-scoped on a newer multi-league database. The report therefore warns against pooling those results directly.

Other caveats

- Autonomy result is partly expected: templates are reliable within coverage and cannot generalise beyond their catalogue; the less obvious finding is strong foundation-model sensitivity.
- Automatic scoring can penalise substantively correct answers because of units or markdown, so reported scores are programmatic-match rates rather than perfect human correctness judgments.
- Memory accuracy benefit remains unproven; SQL caching needs explicit safety guards; correction memory can also conflate transient API errors with genuine semantic failures.
- The project is text-only with a limited tool catalogue; multimodal evidence and broader tools are natural extensions.

Final interpretation

The project begins with the expectation that multi-agent orchestration may be the main source of improved football analytics. The experiments instead point to a more practical hierarchy of priorities. First, build a clean and well-documented data layer with canonical entities, stable serving views and explicit schema semantics. Second, give the LLM safe tools that perform retrieval and computation. Third, add orchestration when the workload is ambiguous, compositional or operationally benefits from modularity and error isolation. For predictable single-step queries, one tool-using LLM is cheaper and just as accurate in this benchmark.

MAS may still be justified for engineering robustness: inspectable control flow, modular debugging, the ability to replace one agent without disturbing the rest, and clearer containment of failures. Memory should be evaluated by what it actually contributes - usually cost amortisation and conversational continuity rather than guaranteed accuracy. Prediction should be delegated to calibrated models and returned as full probability distributions, not just an argmax label.

Best one-sentence takeaway: clean structured data and good tools matter more than simply adding more agents; orchestration should be paid for only when the workload or engineering requirements genuinely need it.